

Lab2_Report_RegisterFile

컴퓨터소프트웨어학부 2021026108 김민제

구현 환경

운영체제 : macOS Sonoma

verilog : iverilog

vcd : gtkwave

cpp : Apple clang version 16.0.0 (clang-1600.0.26.6)

지난 과제 파일

ALU.cpp ALU.h ALU.v 는 지난 과제 제출과 동일한 파일을 사용하였습니다.

전체 구조 분석 및 설계

지난 과제로 만든 ALU 에 새로운 과제인 Register File 을 완성하여 TOP 이라는 파일에 두 모듈을 합성하여 하나의 모듈처럼 만드는 것이 목적입니다.

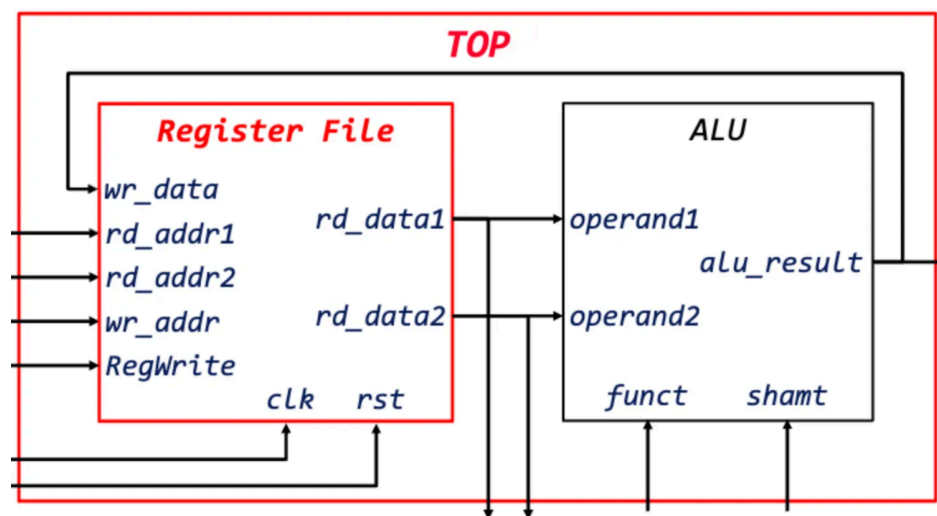
먼저

Register File 의 input 을 확인해보면

- `wr_data` : ALU 의 계산 결과를 input으로 받아 저장합니다.
- `rd_addr1` `rd_addr2` : 읽어올 레지스터의 주소를 입력 받습니다.
- `wr_addr` : 계산 결과를 저장할 주소를 받습니다.
- `RegWrite` : 값을 저장할지 말지 제어하는 신호입니다.
- `clk` : 변화할 때 마다 `RegWrite` 에 따라 값을 저장합니다.
- `rst` : Register를 초기화 시켜줄 지 정하는 신호로 이번 과제에서는 random으로 값을 초기화 시킵니다.

Output 을 확인해보면

- `rd_data1` `rd_data2` : 입력받은 주소를 바탕으로 읽어온 레지스터의 값을 ALU의 `operand` 로 넘겨줍니다.



CPP 설계 및 구현

1. RF.cpp

1. RF::read 함수 구현

함수 인자로 레지스터의 주소를 전달받아 해당 레지스터의 값을 `rd_data` 를 통하여 내보냅니다.

```
void RF::read(uint32_t rd_addr1, uint32_t rd_addr2, uint32_t *rd_data1,
              uint32_t *rd_data2) {
    *rd_data1 = register_files[rd_addr1];
    *rd_data2 = register_files[rd_addr2];
}
```

이때 `register_files` 에 존재하지 않는 주소로 입력이 들어오는 경우가 발생하는 경우에 대해서 처리하는 방식을 고민하였는데 Trouble Shooting에서 자세히 작성했습니다.

2. RF::write 함수 구현

`RegWrite` 이 1 일때 만 레지스터의 결과를 저장해야하므로 if문을 사용하여 구현하였습니다.

```
void RF::write(uint32_t wr_addr, uint32_t wr_data, uint32_t RegWrite) {
    if (RegWrite) {
        register_files[wr_addr] = wr_data;
    } else if (wr_addr == 0) { // 리포트 작성중 추가 - Trouble Shooting 2번
        register_files[wr_addr] = 0; //$zero 레지스터값 유지
    }
}
```

이 때 역시 `wr_addr` 의 주소의 범위를 고려하는 방식에 관해 고민했는데 Trouble Shooting에 설명해주었습니다.

2. TOP.cpp

1. TOP::tick 함수 구현

이 파일은

`ALU` 와 `RF` 모듈을 하나로 합쳐주는 부분이므로 위 모듈 설계도처럼 먼저 `RF` 에서 `read` 를 하고 `read` 로 읽어온 값을 `ALU` 의 `operand` 로 보내 `compute` 해주고 그 계산 값을 다시 `RF` 에 `write` 해주도록 설계하면 되서 순차적으로 `rf.read` , `alu.compute` , `rf.write` 을 실행해주면 됩니다.

```
void TOP::tick(uint32_t rd_addr1, uint32_t rd_addr2,
               uint32_t wr_addr, uint32_t shamt, uint32_t aluop, uint32_t RegWrite,
               uint32_t *rd_data1, uint32_t *rd_data2, uint32_t *wr_data) {

    rf.read(rd_addr1, rd_addr2, rd_data1, rd_data2);
    alu.compute(*rd_data1, *rd_data2, shamt, aluop, wr_data);
    rf.write(wr_addr, *wr_data, RegWrite);
}
```

Verilog 설계 및 구현

1. RF.v

1. Fill in the asynchronous functions

`read` 는 asynchronous하게 동작하므로 input인 `Register Address` 가 변경되자마자 동작해야한다.

```
always @(*) begin
    rd_data1 = register_file[rd_addr1];
    rd_data2 = register_file[rd_addr2];
end
```

2. FILL what happens synchronously

`write` 은 positive edge가 될 때마다 동작해야 하므로 `clk` 이 변할때 마다 동작하게 구현해야 합니다.

```
always @(posedge clk) begin
    if (rst) begin // 리셋 시 실행되는 코드
        $readmemh("../cpp/initial_reg.mem", register_file);
        // FILL what happens synchronously
    end
    else if (RegWrite) begin
        // register_file[wr_addr] <= wr_data;
        // 초기 코드 $zero register 미고려
        if (wr_addr)
            register_file[wr_addr] <= wr_data;
        else
            register_file[wr_addr] <= 0;
    end
end
```

2. TOP.v

1. 만든 `ALU` 와 `RF` 모듈을 연결해야 하므로 모듈을 인스턴스화 해주고 각 input과 output들을 연결해주면 됩니다.
2. `RF` 의 경우 테스트벤치로부터 `clk` , `rst` , `rd_addr1` , `rd_addr2` , `RegWrite` 를 전달받습니다.

```
RF rf (
    .clk(clk),
    .rst(rst),
    .rd_addr1(rd_addr1),
    .rd_addr2(rd_addr2),
    .rd_data1(rd_data1),
    .rd_data2(rd_data2),
    .RegWrite(RegWrite),
    .wr_addr(wr_addr),
    .wr_data(wr_data)
);
```

3. ALU의 경우 RF에게 rd_data1, rd_data2를 각각 operand1, operand2로 받고 테스트벤치로부터 shamt, funct를 입력받습니다. 그리고 입력받은 값들을 연산하여 그 결과를 다시 RF에게 전달해주도록 구현했습니다.

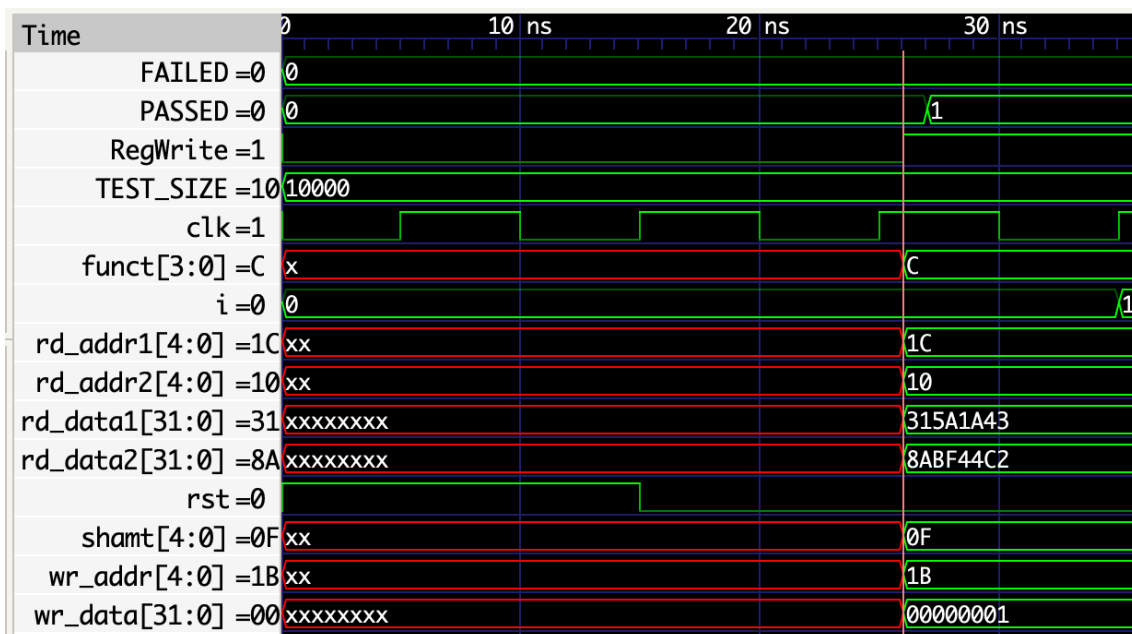
```
ALU alu (
    .operand1(rd_data1),
    .operand2(rd_data2),
    .shamt(shamt),
    .funct(funct),
    .alu_result(wr_data)
);
```

Result & vcd File

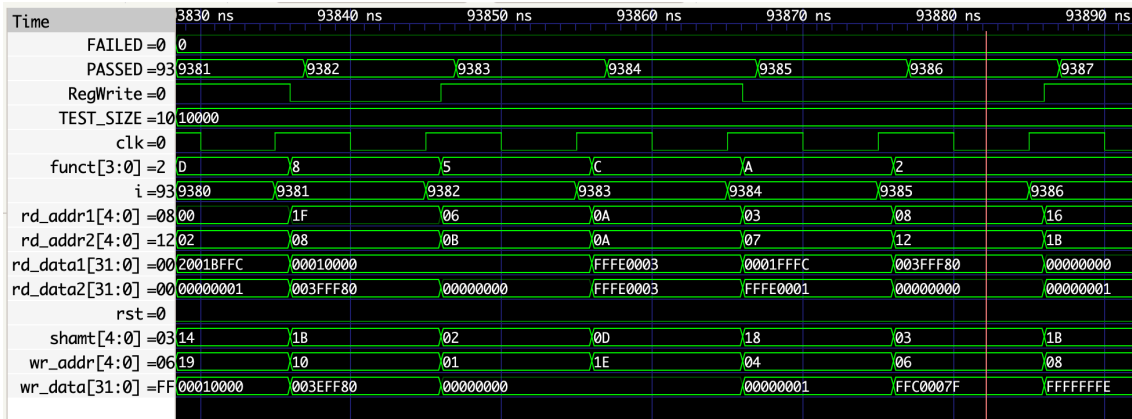
테스트 케이스를 10000번을 잡아서 테스트를 돌린 결과입니다. 테스트 케이스는 cpp파일의 main.cpp에서 랜덤하게 만든 .mem 파일들을 읽어와서 테스트를 돌린 결과 10000번 모두 PASSED로 잘 동작하였습니다. 이때 TOP_tb.v와 RF.v의 .mem 파일을 읽어오는 경로를 ../cpp/로 바꾸어 편리하게 값을 읽었습니다.

```
PASSED
PASSED
PASSED = 10000, FAILED = 0
TOP_tb.v:126: $finish called at 100035000 (1ps)
```

프로그램 동작 초반 rst = 1 일때 초기화 하고 rst = 0 으로 바뀌며 테스트케이스들을 돌릴 준비를 하는 과정입니다.



프로그램 동작 중반에 값을 읽어오고 저장하는 과정들의 값들의 변화입니다. RegWrite의 값에 따라 적절하게 Register에 값을 저장하는 것을 확인할 수 있습니다.



Trouble Shooting

1. 레지스터 파일 주소 접근 오류 처리

Register File은 특정 주소를 통해 데이터를 읽고 쓰는 구조를 가집니다. 하지만 잘못된 주소에 접근하면 예기치 않은 동작이 발생할 수 있으므로, 이를 방지하는 방법이 필요하다고 생각했습니다. 그래서 처음에는 예외처리를 하여 주소 범위를 넘으면 0을 넣어주는 방식으로 코드를 짰는데 처음에 Register를 random한 값으로 초기화해주기 때문에 함부로 0을 넣으면 안된다는 것을 깨달았습니다. 이 프로그램의 경우 32 미만의 값만 받도록 설정되어 있어 초기화하는 값을 설정하는 방식도 모호하여 그냥 무시하였습니다. 아래는 verilog로 범위를 설정한 예시입니다.

```
# read
// if (rd_addr1 <= 5'b11111)
rd_data1 = register_file[rd_addr1];
// if (rd_addr2 <= 5'b11111)
rd_data2 = register_file[rd_addr2];
# write
//end else if (RegWrite && wr_addr != 5'b00000)
```

2. \$zero 레지스터의 값(0) 유지 문제

처음에는 \$zero 레지스터를 고려하지 않고 그냥 write을 하여 \$zero 레지스터는 항상 0을 유지해야 한다는 조건을 놓쳤습니다. 리포트를 작성하다 특수한 레지스터의 경우 값을 보존해야해서 그 코드를 추가했습니다.

```
} else if (wr_addr == 0) {
    register_files[wr_addr] = 0; //$zero 레지스터값 유지
}
```

3. 리셋과 쓰기가 동시에 일어나는 경우는 없을까?

초기에 코드를 작성하는 도중에 값을 읽어오는 부분을 잘못 작성하여 이상한 값을 읽어오는 이유로 FAILED가 계속 나타났는데 이 이유가 타이밍이 이상하여 다르게 동작하는 것은 아닌가 고민해보았습니다. 그래서 리셋과 쓰기가 동시에 일어나는 경우가 존재한다고 생각했고 이를 찾아보았습니다. 일단 리셋과 쓰기의 경우 positive edge에 따라 상태가 변할 때 write을 하고 초기화는 rst가 변할 때 동작하여 동시에 일어날 수 있으므로 따로 처리를 하지 않으면 동시에 일어날 수 있어 상호 배타적으로 처리되도록 설계해야 합니다. 리셋은 우선순위를 가져 리셋이 발생하면 시스템의 모든 동작이 초기화 상태로 돌아가게 설계가 되어 있거나 리셋이 비활성화 된 경우에만 쓰기를 허용해야 합니다. 그래서 처음에 주석처리한 부분으로 작성했다. 아래 코드로 변했습니다.

```
always @(posedge clk) begin
if (rst) begin // 리셋 시 실행되는 코드
    $readmemh("../cpp/initial_reg.mem", register_file);
    // FILL what happens synchronously
    end
    //if (RegWrite) begin
else if (RegWrite) begin
    if (wr_addr)
        register_file[wr_addr] <= wr_data;
    else
        register_file[wr_addr] <= 0;
    end
end
end
```